

Welcome to the Data Quality Workshop!

While we're waiting for folks to join, feel free to get started on setup:

1. Sign up for a workshop sandbox

- a. Important:
Log out of dbt if you are in an existing account.
- b. Sign up for the workshop (don't forget to save the un/pw!):
URL: <http://workshops.us1.dbt.com/workshop>
WORKSHOP: Mastering Data Quality with dbt
PASSCODE: Summit2026!

2. Setup in platform:

- a. Go to Projects > Analytics, then launch dbt Studio
- b. Create a branch named [feature/<your_full_name>_training](#)
- c. In the command line run 'dbt build'

Mastering Data Quality with dbt

The tools for ensuring your data is complete, accurate, and useful



Carol Ohms

Senior Resident Architect
dbt Labs



Stephen Thibeault

Senior Resident Architect
dbt Labs

1

Setup

Time to set up your sandbox for hands-on exercises

2

Section 1: Data Tests

How to leverage testing right out of the box

3

Section 2: Custom Tests

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions

Workshop Norms

Commit 100%

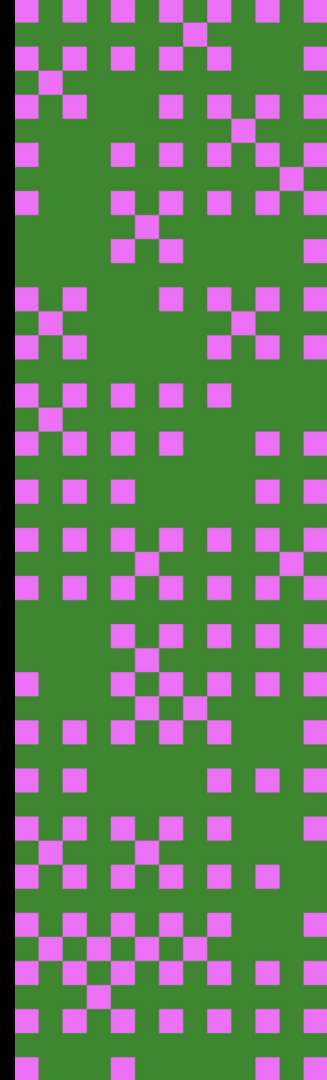
Following along with the exercises is the best way to solidify what you're learning!

Be curious!

We love a collaborative environment - asking questions is a great way for everyone to learn.

Leverage Community

Help each other out - answer questions, weigh in, and share your experiences!



Workshop Structure

Instructor-led content

- + **Introduction**
to new dbt concepts
- + **Walk through**
of implementation details
- + **Check understanding**
with section checkpoints

Hands-on Exercises

- + **Solidify learning**
through implementation
- + **Get past learning curves**
with instructor guidance
- + **Enhance knowledge**
with collaborative
discussions

Setup

05:00

While we're waiting for folks to join, feel free to get started on setup:

1. Sign up for a workshop sandbox

- a. Important:
Log out of dbt if you are in an existing account.
- b. Sign up for the workshop (don't forget to save the un/pw!):
URL: <http://workshops.us1.dbt.com/workshop>
WORKSHOP: Mastering Data Quality with dbt
PASSCODE: Summit2026!

2. Setup in platform

- a. Go to Projects > Analytics, then launch dbt Studio
- b. Create a branch named [feature/<your_full_name>_training](#)
- c. In the command line run 'dbt build'



Data Quality

Why should we test our data?

Imagine

you're a data analyst working for Jaffle Shop. Your dashboards surface trends that drive important decisions like:

- ordering inventory
- starting and ending sales campaigns

!! Suddenly, an issue occurs

On Monday, your sales campaign dashboard shows an unusually drastic drop in sales over the weekend, indicated by these anomalies:

- lower sales figures than known trends
- incomplete customer transactions
- stagnant inventory levels

Why should we test our data?

You identify the root cause

A recent system update introduced bugs which resulted in impacts to your data:

- Missing transaction fields
- Duplicate sales entries
- Failed sales updates

However, there are still lasting impacts

- Marketing campaigns that ended prematurely
- Inventory orders that were miscalculated
- Executive reports that gave misleading information

What do we want?

Data quality tests!

Tests are how we can protect our data's integrity through early detection, ensuring that our data products are more accurate and trusted *before* consumption.

We should be testing the assumptions we make frequently, such as:

- our rows aren't duplicated
- important columns are populated or have expected values
- entities we work with have valid records in another table
- the data we're working with is up-to-date
- the logic we're writing or rewriting produces reliable results
- etc...

Introducing data quality

...and we should be testing it along our data pipelines:

- Testing ingested data
Ensuring the structure and freshness we expect, before any further transformations take place
- Testing our transformed data
Ensuring our further wrangling and reshaping of the data produces expected outputs and meets expectations downstream

1

Setup

Time to set up your sandbox for hands-on exercises

2

Section 1: Data Tests

How to leverage testing right out of the box

3

Section 2: Custom Tests

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions



Data Tests

What are data tests in dbt?

Configurations to make assertions about source data or model results.

In plain SQL, you'd likely write a query that *returns anomalies* when you're checking for bad data, such as:

```
select * from my_table
qualify count(*) over(
  partition by id
) > 1
```

*If no rows are returned, your test **passes***

In dbt, data tests are configured in source or model YML:

```
models:
  - name: my_table
    columns:
      - name: id
        data_tests:
          - unique
```

behind the scenes, the SQL for the test is constructed similarly.

Built-in dbt data tests

dbt has **four** data tests that are ready to use out of the box:

- unique
- not_null
- accepted_values
- relationships

Built-in dbt data tests

dbt has **four** data tests that are ready to use out of the box:

- **Unique**
 - *Asserts that every value in a column is unique across all records.*
- not_null
- accepted_values
- relationships

```
1  # _models.yml
2
3  models:
4    - name: stg_jaffle_shop__products
5      description: Standardized product data
6      columns:
7        - name: product_id
8          data_tests:
9            - unique
10
```

Built-in dbt data tests

dbt has **four** data tests that are ready to use out of the box:

- unique
- **Not_null**
 - *Asserts that every value in a column is not NULL.*
- accepted_values
- relationships

```
1  # _models.yml
2
3  ∨ models:
4  ∨   - name: stg_jaffle_shop__products
5      description: Standardized product data
6  ∨   columns:
7  ∨       - name: product_id
8  ∨       data_tests:
9           - unique
10          - not_null
```

Built-in dbt data tests

dbt has **four** data tests that are ready to use out of the box:

- unique
- Not_null
- **Accepted_values**
 - *Asserts that every value in a column matches one in a given list.*
- relationships

```
1  # _models.yml
2
3  models:
4    - name: stg_jaffle_shop__orders
5      description: Standardized order data
6      columns:
7        - name: customer_id
8        - name: status
9      data_tests:
10        - accepted_values:
11          values:
12            - pending
13            - complete
14            - cancelled
15
```

Built-in dbt data tests

dbt has **four** data tests that are ready to use out of the box:

- unique
- Not_null
- accepted_values
- **Relationships**
 - *Asserts that every value in a column can match to the value of a column of another table.*

```
1  # _models.yml
2
3  models:
4    - name: stg_jaffle_shop__orders
5      description: Standardized order data
6      columns:
7        - name: customer_id
8          data_tests:
9            - relationships:
10              to: ref('stg_jaffle_shop__customers')
11              field: customer_id
12        - name: status
```

Hands-on Exercise 1:

Data Tests

05:00

We want to ensure the `customer_lifetime_values` model supports our expectation that each `customer_id` corresponds to a single, unique, and identifiable row in the results.

1. Open the `_workshop_resources > exercises > exercise_1.md` file.
2. Follow the instructions to test our assertions.

Checkpoint: Data Tests

- What are the four native dbt data tests?
- What are some data quality issues we might test for?
- Which checks are a dbt best practice min. for testing?

1

Setup

Time to set up your sandbox for hands-on exercises

2

~~Section 1:~~ ~~Data Tests~~

How to leverage testing right out of the box

3

Section 2: Custom Tests

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions



Custom Data Tests

What are **custom data tests** in dbt?

Assertions made by writing our own logic to test source data or model results.

These are helpful when a native dbt test doesn't cover the assertion you want to make. There are two types:

- Singular tests
- Generic tests



Singular Tests

Singular tests

Tests which are written to apply to specific models or sources.

```
1  # tests/greater_than_zero.sql
2
3  select
4      order_total
5  from {{ ref('fct_orders') }}
6  where order_total <= 0
7
```

Singular tests

Tests which are written to apply to specific models or sources.

Explanation:

- SQL files
- Written using direct `{{ ref }}`s or `{{ source }}`s, just like a model.
- Located in the `/tests/` folder
- dbt understands which model the test belongs to through the use of the `{{ ref }}`s or `{{ source }}`es used.

```
1  # tests/greater_than_zero.sql
2
3  select
4      order_total
5  from {{ ref('fct_orders') }}
6  where order_total <= 0
7
```

Singular tests: Extending

What if we want to apply the same test to other models?

```
1  # tests/greater_than_zero.sql
2
3  select
4    |    order_total
5  from {{ ref('fct_orders') }}
6  where order_total <= 0
7
```

Singular tests: Extending

What if we want to apply the same test to other models?

+ New file > copy > paste > change

```
1  # tests/greater_than_zero.sql
2
3  select
4      order_total
5  from {{ ref('fct_orders') }}
6  where order_total <= 0
7
```

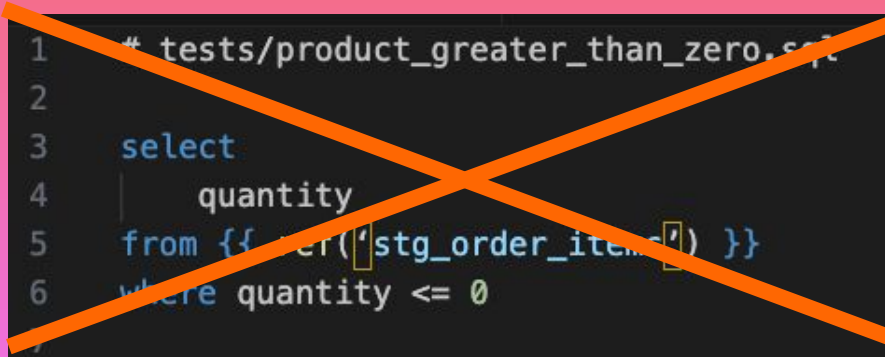
```
1  # tests/product_greater_than_zero.sql
2
3  select
4      quantity
5  from {{ ref('stg_order_items') }}
6  where quantity <= 0
7
```

Singular tests: Extending

What if we want to apply the same test to other models?

~~+ New file → copy → paste → change~~

```
1  # tests/greater_than_zero.sql
2
3  select
4      order_total
5  from {{ ref('fct_orders') }}
6  where order_total <= 0
7
```



```
1  # tests/product_greater_than_zero.sql
2
3  select
4      quantity
5  from {{ ref('stg_order_items') }}
6  where quantity <= 0
7
```




Generic Tests

Generic tests

Tests which are written for any given models or sources.

```
1  # tests/generic/greater_than_zero.sql
2
3  {% test greater_than_zero(model, column_name) %}
4
5  select
6      {{ column_name }}
7  from {{ model }}
8  where {{ column_name }} <= 0
9
10 {% endtest %}
11
```

Promoting a singular test

Recap

```
1  # tests/greater_than_zero.sql
2
3  select
4      order_total
5  from {{ ref('fct_orders') }}
6  where order_total <= 0
7
```

Promoting a singular test

Explanation

1. Create a file in the `tests/generic/` folder, add your SQL

```
1  # tests/generic/product_greater_than_zero.sql
2
3  select
4      quantity
5  from {{ ref('stg_order_items') }}
6  where quantity <= 0
7
```

Promoting a singular test

Explanation

1. Create a file in the `tests/generic/` folder, add your SQL
2. Wrap your statement in a `test` wrapper
- 3.

```
1  # tests/generic/product_greater_than_zero.sql
2
3  {%- test greater_than_zero() %}
4
5  select
6      | order_total
7  from {{ ref('fct_orders') }}
8  where order_total <= 0
9
10 {%- endtest %}
11
```

Promoting a singular test

Explanation

1. Create a file in the `tests/generic/` folder, add your SQL
2. Wrap your statement in a `test` wrapper
3. Add params for `model` (req'd) and `column_name` (opt'l)

```
1  # tests/generic/product_greater_than_zero.sql
2
3  {% test greater_than_zero(model, column_name) %}
4
5  select
6      | order_total
7  from {{ ref('fct_orders') }}
8  where order_total <=
9
10  {% endtest %}
11
```

Promoting a singular test

Explanation

1. Create a file in the `tests/generic/` folder, add your SQL
2. Wrap your statement in a `test` wrapper
3. Add params for `model` (req'd) and `column_name` (opt'l)
4. Change the model and columns to use the param `variables`

```
1  # tests/generic/product_greater_than_zero.sql
2
3  {% test greater_than_zero(model, column_name) %}
4
5  select
6      | {{ column_name }}
7  from {{ model }}
8  where {{ column_name }} <= 0
9
10  {% endtest %}
11
```

Promoting a singular test

Explanation

1. Create a file in the `tests/generic/` folder, add your SQL
2. Wrap your statement in a `test` wrapper
3. Add params for `model` (req'd) and `column_name` (opt'l)
4. Change the model and columns to use the param `variables`
5. Add the test to a model's YML!

```
1  # _models.yml
2
3  ∨ models:
4  ∨   - name: fct_orders
5  ∨     columns:
6  ∨       - name: order_total
7  ∨       data_tests:
8  ∨         - greater_than_zero
9
```


What if you need more than **model**
or **column_name**?

You can add more params!

Adding new parameters

**Change the test to
include a new
parameter**

tests/generic/greater_than_zero.sql

```
{%- test greater_than_zero(model, column_name, threshold=0) %}  
select  
    {{ column_name }}  
from {{ model }}  
where {{ column_name }} <= 0  
{%- endtest %}
```

Adding new parameters

**Template the spot
you want the
parameter to
populate**

tests/generic/greater_than_zero.sql

```
{%- test greater_than_zero(model, column_name, threshold=0) %}  
select  
    {{ column_name }}  
from {{ model }}  
where {{ column_name }} <= {{ threshold }}  
{%- endtest %}
```

Adding new parameters

**Make sure your
test and file
name reflect the
test still!**

tests/generic/greater_than.sql

```
{%- test greater_than(model, column_name, threshold=0) %}  
select  
    {{ column_name }}  
from {{ model }}  
where {{ column_name }} <= {{ threshold }}  
{%- endtest %}
```

Adding new parameters

Change the test
reference in your
YML

_models.yml

```
models:
  - name: fct_orders
    columns:
      - name: order_total
        data_tests:
          - greater_than
```

Adding new parameters

Now that you
have more
parameters, we
need a colon after
the test

_models.yml

```
models:
  - name: fct_orders
    columns:
      - name: order_total
        data_tests:
          - greater_than:
```

Adding new parameters

**Specify the
parameter name
and argument
value**

_models.yml

```
models:
  - name: fct_orders
    columns:
      - name: order_total
        data_tests:
          - greater_than:
              arguments:
                threshold: 1000
```

Hands-on Exercise 2:

Custom Generic Tests

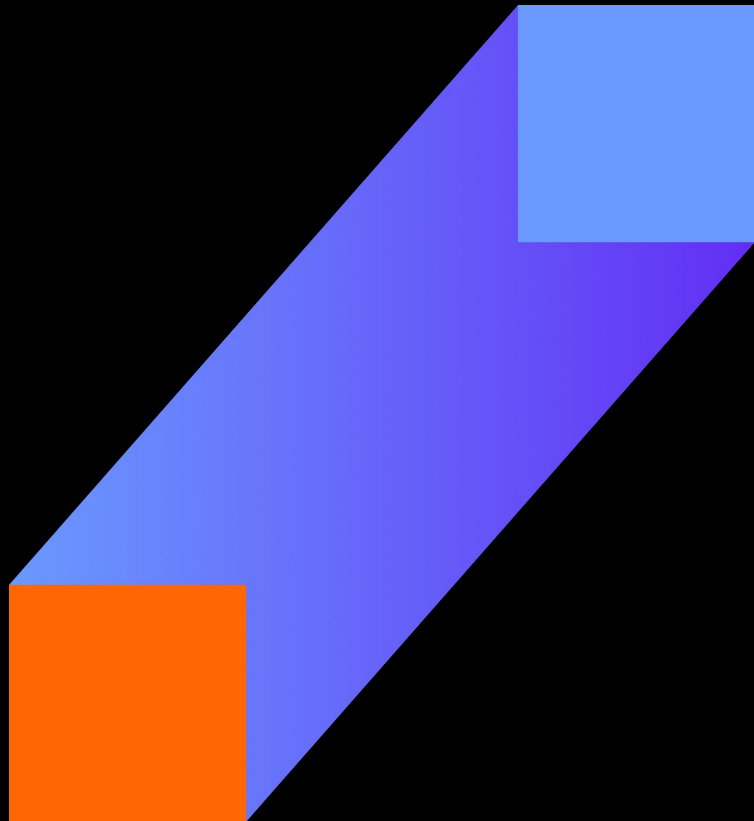
05:00

We want to ensure the `order_total` column on our `fct_orders` model meets our expectation that the `data_type` is `DECIMAL`.

1. Open the `_workshop_resources > exercises > exercise_2.md` file.
2. Follow the instructions to create a custom generic test.

Bonus: Package Tests

more generic tests!



Building tests from scratch?

In most cases, you'll be able to find a package test that does what you need!

Popular packages with tests:

- **dbt_utils**
A package of useful functions. It also includes some useful generic tests, such as one to test uniqueness on a combination of columns.
- **dbt_expectations**
A dedicated generic test suite inspired by the Great Expectations package for Python.
- **dbt_meta_testing**
Helps you flag when models don't meet customizable testing minimums (such as a unique and not_null test on the primary key).

The best part? It only takes a matter of minutes!

Implementing package tests

1. Go to hub.getdbt.com and click on a package.
2. Copy the installation config.
3. Go to Studio and create a `packages.yml` file at the root.
4. Paste the config, save.
5. Execute `dbt deps` from the command bar to install the package.

Nice! You can now read through the package's documentation to find examples on how to use their generic tests.

Checkpoint: Custom Data Tests

- **What scenario is best for creating a singular test?**
- **What scenario is best for creating a generic test?**
- **What are some components/requirements of a generic test?**
- **How are generic tests made available beyond a single project?**

1

Setup

Time to set up your sandbox for hands-on exercises

2

~~Section 1:~~ ~~Data Tests~~

How to leverage testing right out of the box

3

~~Section 2:~~ ~~Custom Tests~~

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions



Model Contracts & Constraints

What are **model contracts** in dbt?

Explicit guarantees about a model's shape, column names, order, and data types, that dbt checks before the model is allowed to build.

Note: Contracts are considered “Pre-flight” checks in dbt, running before a model is actually built, different than data tests, which run after.

- Every column must be declared with a name and data_type once a contract is enforced.
- If the model's actual output doesn't match, the build fails instead of silently shipping a different structure downstream.

Why use model contracts?

Are they just tests? Why are they different? What do they unlock?

- Contracts exist to guarantee your data's shape to downstream consumers

Highly recommended for any **public** models

- Public models are those that can be used by downstream projects
- Because they run before a model is built, breaking changes will not flow downstream
- They also run automatically anytime the model is ran, unlike traditional dbt data tests

Building a contract

What is required to define a contract in dbt?

Building a contract

What is required to define a contract in dbt?

- Contracts must be listed as "enforced" in the model yaml

```
1  # models/marts/docs/_dim_customers.yml
2
3  models:
4    - name: dim_customers
5      config:
6        contract:
7          enforced: true
8
```

Building a contract

What is required to define a contract in dbt?

- Contracts must be listed as “enforced” in the model YAML
- Once marked as enforced the column names and data types defined in the YAML will be tested

NOTE: All columns in the table need to be included.

```
1  # models/marts/docs/_dim_customers.yml
2
3  models:
4    - name: dim_customers
5      config:
6        contract:
7          enforced: true
8        columns:
9          - name: customer_id
10            data_type: int
11          - name: customer_email
12            data_type: varchar
13
```

What happens when a contract breaks?

Without a contract, a renamed/retyped column ships quietly and breaks things downstream.

With a contract, the build fails immediately with a clear mismatch error

dim_customers

0.29s

Filter Logs

Debug logs

16:20:41 Failure in model dim_customers (models/dim_customers.sql)

16:20:41 Compilation Error in model dim_customers (models/dim_customers.sql)

This model has an enforced contract that failed.

Please ensure the name, data_type, and number of columns in your contract match the columns in your model's definition.

column_name	definition_type	contract_type	mismatch_reason
CUSTOMER_EMAIL	TEXT	FIXED	data type mismatch

What are **constraints** in dbt?

Additional rules layered on top of a contract that describe expectations about the data itself.

You will notice that these are very similar to dbt data tests, but they are ran as at DDL time rather than after a build.

- Enforcement depends on the warehouse you are using
- Because of the lack of enforcement on many warehouses, these are often used more for information rather than expected to be enforced.

Available constraints

There are five primary constraints in dbt
(check docs for enforcement)

- not_null
- unique
- primary_key
- foreign_key
- check

Available constraints

There are five primary constraints in dbt (check docs for enforcement)

- Not_null
 - *Asserts that every value in a column is not NULL.*
- Unique
- primary_key
- foreign_key
- check

```
1  # models/marts/docs/_dim_customers.yml
2
3  models:
4    - name: dim_customers
5      config:
6        contract:
7          enforced: true
8      columns:
9        - name: customer_id
10          data_type: int
11          constraints:
12            - type: not_null
13
```

Available constraints

There are five primary constraints in dbt (check docs for enforcement)

- not_null
- Unique
 - *Asserts that every value in a column is unique across all records.*
- primary_key
- foreign_key
- check

```
1  # models/marts/docs/_dim_customers.yml
2
3  models:
4    - name: dim_customers
5      config:
6        contract:
7          enforced: true
8        columns:
9          - name: customer_id
10            data_type: int
11            constraints:
12              - type: not_null
13              - type: unique
14
```


Available constraints

There are five primary constraints in dbt (check docs for enforcement)

- unique
- not_null
- Primary_key
 - *Combines both the unique and not_null constraints into the same check*
- foreign_key
- check

```
1  # models/marts/docs/_dim_customers.yml
2
3  models:
4    - name: dim_customers
5      config:
6        contract:
7          enforced: true
8        columns:
9          - name: customer_id
10            data_type: int
11            constraints:
12              - type: primary_key
13
```

Available constraints

There are five primary constraints in dbt (check docs for enforcement)

- unique
- not_null
- primary_key
- Foreign_key
 - Asserts that every value in a column can match to the value of a column of another table.
- check

```
1  # models/marts/docs/_dim_customers.yml
2
3  models:
4    - name: dim_customers
5      config:
6        contract:
7          enforced: true
8      columns:
9        - name: customer_id
10          data_type: int
11          constraints:
12            - type: primary_key
13        - name: customer_top_product_id
14          data_type: int
15          constraints:
16            - type: foreign_key
17              to: ref('dim_product')
18              to_columns: [product_id]
```

Available constraints

There are five primary constraints in dbt
(check docs for enforcement)

- unique
- not_null
- primary_key
- foreign_key
- Check
 - Asserts that a custom defined boolean expression evaluates to *TRUE*

```
1 # models/marts/docs/_dim_customers.yml
2
3 models:
4   - name: dim_customers
5     config:
6       contract:
7         enforced: true
8     columns:
9       - name: customer_id
10         data_type: int
11         constraints:
12           - type: primary_key
13       - name: customer_top_product_id
14         data_type: int
15         constraints:
16           - type: foreign_key
17             to: ref('dim_product')
18             to_columns: [product_id]
19       - name: lifetime_sale_amt
20         data_type: int
21         constraints:
22           - type: check
23             expression: ">= 0"
```

Hands-on Exercise 3:

Contracts and Constraints

05:00

We want to ensure the `order_total` column on our `fct_orders` model meets our expectation that the `data_type` is `DECIMAL`.

1. Open the `_workshop_resources > exercises > exercise_3.md` file.
2. Follow the instructions to add a contract.

Checkpoint: Contracts and Constraints

- **What's the difference between a contract and a test?**
- **What is important to remember about the enforcement of constraints?**
- **When do contracts run when creating a model?**

1

Setup

Time to set up your sandbox for hands-on exercises

2

Section 1: Data Tests

How to leverage testing right out of the box

3

Section 2: Custom Tests

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions



Unit Tests

What are **unit tests** in dbt?

Assertions that, given certain data scenarios, our logic outputs expected results. Unit tests run before your model, which is skipped upon test failure.

Here are different case statements which classify customers into tiers:

```
case
  when spend >= 1000 then "gold"
  when spend >= 500 then "silver"
  when spend >= 100 then "bronze"
  else NULL
end as customer_tier
```



```
case
  when coalesce(spend, 0) < 100 then NULL
  when spend >= 1000 then "gold"
  when spend >= 500 then "silver"
  else "bronze"
end as customer_tier
```



```
case
  when spend = 0 then NULL
  when spend >= 100 then 'bronze'
  when spend >= 500 then 'silver'
  else 'gold'
end as customer_tier
```



In these simple examples, it's clear which will work – however, that can become more unclear as business rules or scenarios arise that we need to account for.

Unit tests are designed to help you ensure reliability of your logic. But before we dive in, let's think about how generic and singular tests work in relation.

Unit vs. Generic vs. Singular tests in dbt

Native and package tests are more specific to one aspect of the results.

For example, they can tell us things like:

- whether we have anomalies in a customer's spend value
- whether the statuses assigned are from a specific list of values
- whether tier totals fit thresholds based on our known market trends

but we still won't know if the customer tiers assigned are correct based on spend.

What about a singular test?

Writing the testing logic for this particular scenario presents an issue of circular reasoning: you're relying on yourself as the point of truth for both the transformation and testing logic - it's like using the same calculator to check if the calculator's results are correct.

Unit Test Implementation

Unit tests are configured via YAML, where mocked data is supplied for both input ("given") and output ("expect")

Unit Test Implementation

First, we define a test under the `unit\_tests` yml key.

- Give the test a descriptive name
- Tell it which model we're testing

```
unit_tests:  
  - name: expect_customer_statuses_to_fit_business_definition  
    model: dim_customers
```

Unit Test Implementation

Then, we give it data to use for our scenarios.

We supply mock data for each of the sources or references we use in our model.

```
unit_tests:
  - name: expect_customer_statuses_to_fit_business_definition
    model: dim_customers
  given:
    - input: ref('stg_customers')
      rows:
        - {customer_id: 1}
        - {customer_id: 2}
        - {customer_id: 3}
        - {customer_id: 4}
    - input: ref('fct_orders')
      rows:
        - {customer_id: 1, amount_usd: 1000}
        - {customer_id: 2, amount_usd: 999}
        - {customer_id: 3, amount_usd: 2}
        - {customer_id: 4, amount_usd: 0}
```

Unit Test Implementation

Finally, we tell it what the results should look like given the data supplied.

```
unit_tests:
  - name: expect_customer_statuses_to_fit_business_definition
    model: dim_customers
    given:
      - input: ref('stg_customers')
        rows:
          - {customer_id: 1}
          - {customer_id: 2}
          - {customer_id: 3}
          - {customer_id: 4}
      - input: ref('fct_orders')
        rows:
          - {customer_id: 1, amount_usd: 1000}
          - {customer_id: 2, amount_usd: 999}
          - {customer_id: 3, amount_usd: 2}
          - {customer_id: 4, amount_usd: 0}
    expect:
      rows:
        - {customer_id: 1, status: gold}
        - {customer_id: 2, status: silver}
        - {customer_id: 3, status: bronze}
        - {customer_id: 4, status: NULL}
```

Unit Tests: Common Practices

- **Only running unit tests in development and CI**

Unit tests ensure the reliability of written logic. Code changes won't suddenly make it into a higher environment without going through development or CI testing (given you're following and enforcing a healthy promotion process).

Contrast this with data tests, in which tests can fail without code changes due to anomalies in the underlying data.

- **Use unit tests when you're questioning the reliability of:**

- Regex patterns
- Complex case/when statements
- Date math
- Window functions
- Truncated values

Hands-on Exercise 4:

Unit Tests

05:00

We want to test that a calculated `boolean` column, `is_valid_email`, in `stg_jaffle_world__customers` correctly identifies valid and invalid emails based on known scenarios.

1. Open the `_workshop_resources > exercises > exercise_4.md` file.
2. Follow the instructions to create a unit test for our assertions.

Checkpoint: Unit Tests

- **How are unit tests different in comparison to other types of testing?**
- **What are some pieces of information you need to define a unit test?**

1

Setup

Time to set up your sandbox for hands-on exercises

2

Section 1: Data Tests

How to leverage testing right out of the box

3

Section 2: Custom Tests

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions



Advanced CI

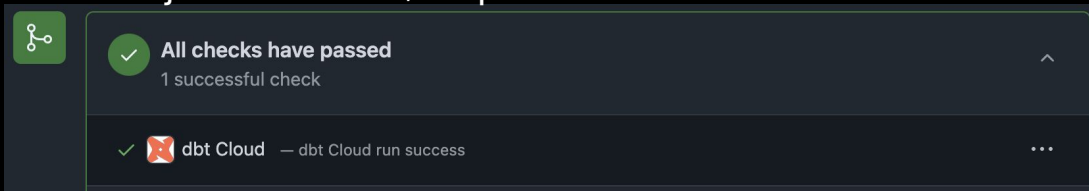
What is **Continuous Integration (CI)** in dbt?

An automated process which builds and tests changes prior to promoting (merging) code to a higher environment.

In dbt, **continuous integration** is a type of job which *listens for new pull requests*. This event triggers the check and reports the status, enabling you to merge with confidence.

How it works:

- A developer opens a pull request
- The action of opening a PR causes a dbt CI job to trigger
- The CI job builds and tests the changed code with a dedicated schema, unique to the PR: **dbt_cloud_pr_<job id>_<pr #>**
- After the job is finished, it updates the PR with the CI status:



What is **Advanced CI**?

An additional option for CI which increases governance and observability for additional reassurance before promotion.

Enabling the **compare changes** feature in CI:

- will use a small subset of data in order to compare the old and new versions of the data in order to surface valuable information such as:
 - changes to row counts
 - changes to the schema (old, new, and modified columns)
 - changes to the primary key
 - data diffs
- will summarize results as a comment on the PR

Advanced CI: Comparison Results

Comparison Overview

Modified

stg_googleheets__global_training_feedback Different

[View in Explorer](#)

Overview

Primary keys

Modified rows

Columns

Columns changed (1)

Column name	State	Value change	Datatype	Column position
INSTRUCTOR_NAME	Modified	✓		

Modified Rows

Overview

Primary keys

Modified rows

Columns

Modified rows

[Show full preview](#)

unique_id	response_row_id	instructor_name	
5bd7a02a3ac9820eda3658736a170f15	30	mtli	Faith Lierheimer
9b93ec8db4c6736bf4780950f57b1948	667	mtli	Dan Horner
1596d8e41a71d6400c8e5b8058d92046	448	mtli	Beth Hipple

Advanced CI: PR Comments

Run Summary



dbt-cloud bot commented 3 days ago

dbt Cloud CI Report

▼ ✓ [CI] Snowflake PR Build

✓ Clone git repository

✓ Create profile from connection Snowflake (override schema to 'dbt_cloud_pr_1862_2802')

⚠ Invoke dbt with `dbt deps`

Comparison Summary

`model.fishtown_internal_analytics.stg_googleheets__global_training_feedback`

Primary keys	Modified rows	Column changes	Row count changes
0 added 0 removed 0 non-unique	54%	1 modified 0 added 0 removed	0%

Advanced CI: Requirements

The **dbt compare** checkbox must be enabled on your CI job.

The base command that runs under the hood is **dbt compare <selection>**.



dbt compare ?

-s state:modified

Compare changes against an environment (Deferral)

QA

Advanced CI: Requirements

If you want the comparison to show modified data and not just modified columns, primary keys must be configured on your models.

Setting a primary key can be done in one of two ways:

Advanced CI:

Requirements

If you want the comparison to show modified data and not just modified columns, primary keys must be configured on your models.

Setting a primary key can be done in one of two ways:

- configuring a primary key constraint

```
1  # models/marts/docs/_fct_orders.yml
2
3  models:
4    - name: fct_orders
5      config:
6        contract:
7          enforced: true
8        columns:
9          - name: order_id
10            data_type: int
11            constraints:
12              - type: primary_key
13
```

Advanced CI: Requirements

If you want the comparison to show modified data and not just modified columns, primary keys must be configured on your models.

Setting a primary key can be done in one of two ways:

- configuring a primary key constraint
- configuring a uniqueness test

```
1  # models/marts/docs/_fct_orders.yml
2
3  models:
4    - name: fct_orders
5      columns:
6        - name: order_id
7          data_tests:
8            - unique
9
10
```

1

Setup

Time to set up your sandbox for hands-on exercises

2

Section 1: Data Tests

How to leverage testing right out of the box

3

Section 2: Custom Tests

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions



Notifications

**Setting up tests alone does not
improve your data quality.**

Errors must be tied to ACTION

Channels for Notifications

The possible channels within dbt for notifications:

- Email
- Slack
- Microsoft Teams
- Webhooks
 - Notification to other systems

Enable all in Account Settings

Job-Level Alerts: Configure

Billing & Usage

Overview

Usage-based features

SSO & SCIM

API tokens

Personal tokens

Service tokens

IP restrictions

Audit log

Webhooks

Your profile

Personal profile

Password & Security

Credentials

CLI

VS Code Extension

Notification settings

Email notifications

Slack notifications

Teams notifications

Job notifications

Configure job completion notification emails. As an account administrator, you may configure notifications for yourself or for other users in your dbt account. You may also configure for external emails in order to integrate with 3rd party systems.

To learn more about configuring dbt notifications, [check out the docs](#).

Notification email

Carol Ohms - carol.ohms@dbtlabs.com

Environment Production

Job name	Succeeds	Warns	Fails	Is canceled
test	☐	☒	☒	☐

Don't want to receive any of these emails? You can unsubscribe from all email notifications by clicking the button below.

 Unsubscribe from all email notifications

Model notifications: **Configure**

You can send email notifications to model owners

1. Enable the configuration in Account Settings → Email notifications
 - a. Set up per Environment for desired outcome(s)

Model notifications

Environment ▾



Enable group/owner notifications on models.

For more information on setting up model level notifications for your account see [docs](#)

☐

Success

☐

Warning

☐

Fails

Model notifications: **YAML** Setup

2. Create ownership using groups in yml

3. Attach group to models in model.yml
or in dbt_project.yml

```
groups:
  - name: finance
    owner:
      # Email is required for model-level
      name: "Finance team"
      email: finance@dbtlabs.com

  # Example of multiple emails supported
  - name: documentation team
    owner:
      name: "Docs team"
      email:
        - docs@dbtlabs.com
        - community@dbtlabs.com
        - product@dbtlabs.com
    config:
      meta:
        slack: '#docs-fox'
```

```
models:
  - name: fact_sales
    description: "Sales data model"
    config:
      group: finance

  - name: dim_campaigns
    description: "Campaigns data model"
    config:
      group: documentation team
```

Checkpoint: Notifications

- **What are the types of notifications available in dbt platform?**
- **Which yaml resource type sets model ownership?**

1

Setup

Time to set up your sandbox for hands-on exercises

2

Section 1: Data Tests

How to leverage testing right out of the box

3

Section 2: Custom Tests

Creating a solution for your own testing scenarios

4

Section 3: Model Contracts & Constraints

Enforcing standards in written code

4

Section 4: Unit Tests

Testing that written logic accomplishes what you need it to

5

Section 5: Advanced CI

Automatic testing to enforce quality checks

6

Section 6: Notifications

Make sure owners know about errors

6

Closing

Resources for next steps and where to ask follow-up questions

Beefing up your Data Quality: Best Practices

1. Ensure all models contain **unique** and not **null tests** on the primary key of the model.
2. Look into available package tests when you need more, such as dbt_utils' **unique_combination_of_columns** test when a surrogate key in the model results isn't ideal.
3. If you can't find a package test that covers your data quality scenario, create a **singular** or **custom generic** test.
4. Use **unit tests** to ensure the integrity of complex logic. Only run unit tests in development and CI.
5. Implement best practice configurations in CI:
 - a. Deferral to limit what needs to be tested or compared
 - b. dbt Compare to surface data diffs for more thorough reviews
 - c. Try limiting dbt Compare to downstream data products (i.e., marts)
 - d. Linting to ensure code quality standards are being followed (In bonus exercise)
6. Enable actions on errors with **notifications**

Thank you for joining!

We want to hear from you:

<https://dbtlearn.typeform.com/dbt-summit-labs>

